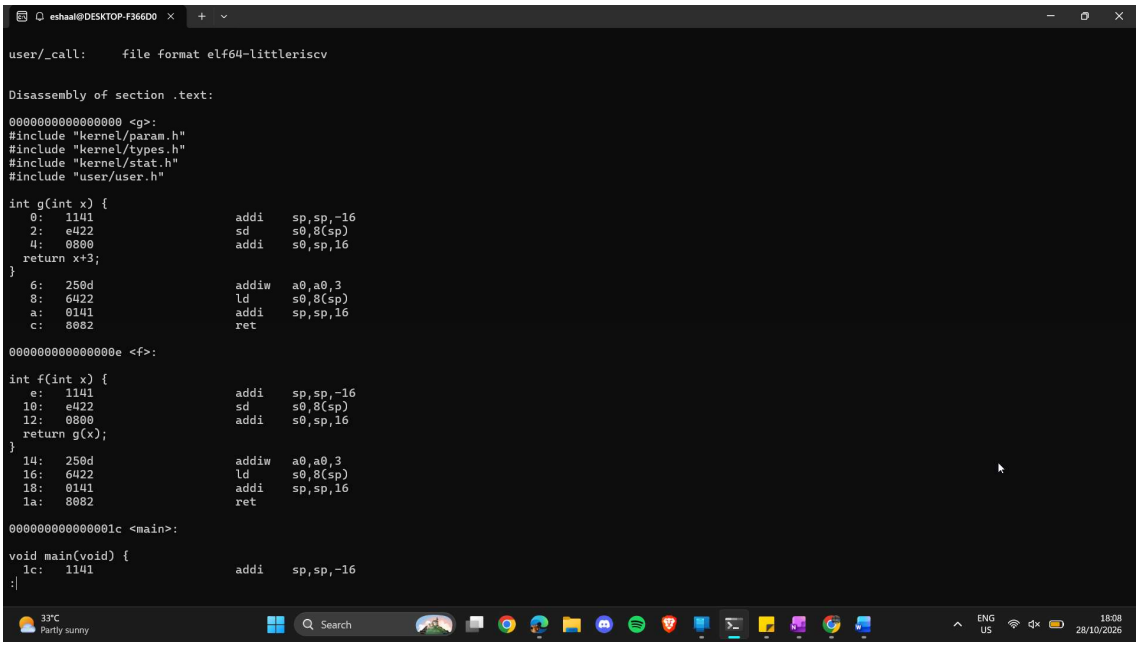
Merging branch:

```
eshaal@DESKTOP-F366D0H:~/OS-Fall26$ cd ~/OS-Fall26/xv6-labs-2026
-bash: cd: /home/eshaal/OS-Fall26/xv6-labs-2026: No such file or directory
eshaal@DESKTOP-F366D0H:~/OS-Fall26$ cd ~/OS-Fall26/xv6-labs-2025
eshaal@DESKTOP-F366D0H:~/OS-Fall26/xv6-labs-2025$ git fetch mit
eshaal@DESKTOP-F366D0H:~/OS-Fall26/xv6-labs-2025$ git checkout -b traps mit/traps
A       fs.img.bk
A       kernel/vm.c.save
A       xv6-labs-2025/kernel/kernel
A       xv6-labs-2025/mkfs/mkfs
A       xv6-labs-2025/user/usys.S
branch 'traps' set up to track 'mit/traps'.
Switched to a new branch 'traps'
```

Finding main, printf, g, f:



Q2.

```
 GNU nano 7.2                        kernel/riscv.h

static inline uint64
r_fp()
{
  uint64 x;
  asm volatile("mv %0, s0" : "=r" (x) );
  return x;
{
```

```
 GNU nano 7.2                         kernel/defs.h *

// printf.c
int             printf(char*, ...) __attribute__ ((format (printf, 1, 2)));
void            panic(char*) __attribute__((noreturn));
void            backtrace(void);
void            printfinit(void);
```

```
 GNU nano 7.2                        kernel/sysproc.c

uint64
sys_pause(void)
{
  backtrace()
  int n;
  uint ticks0;

  argint(0, &n);
  if(n < 0)
    n = 0;
  acquire(&tickslock);
  ticks0 = ticks;
  while(ticks - ticks0 < n){
    if(killed(myproc())){
      release(&tickslock);
      return -1;
    }
    sleep(&ticks, &tickslock);
  }
  release(&tickslock);

  return 0;
}
```

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ bttest
backtrace:
0x0000000080001e2c
0x0000000080001d18
0x0000000080001a9c
$ QEMU: Terminated
eshaal@DESKTOP-F366D0H:~/OS-Fall26$ addr2line -e kernel/kernel

??:0

??:0

??:0
0x0000000080001e2c
/home/eshaal/OS-Fall26/kernel/sysproc.c:73
0x0000000080001d18
/home/eshaal/OS-Fall26/kernel/syscall.c:141 (discriminator 1)
0x0000000080001a9c
/home/eshaal/OS-Fall26/kernel/trap.c:80
```

```
  GNU nano 7.2                        kernel/printf.c *

  if(panicking == 0)
    release(&pr.lock);

  return 0;
}

void
panic(char *s)
{
  panicking = 1;
  printf("panic: ");
  printf("%s\n", s);
  backtrace()
  panicked = 1; // freeze uart output from other CPUs
  for(;;)
    ;
}
```

Compiles:

```
/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stre
ssfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/
_forphan user/_dorphan  user/_call user/_bttest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) bloc
ks 1953 total 2000
balloc: first 973 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -
smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,i
f=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-b
us.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Q3

```
 GNU nano 7.2                          Makefile *
       $U/_forktest\
       $U/_grep\
       $U/_init\
       $U/_kill\
       $U/_ln\
       $U/_ls\
       $U/_mkdir\
       $U/_rm\
       $U/_sh\
       $U/_stressfs\
       $U/_usertests\
       $U/_grind\
       $U/_wc\
       $U/_zombie\
       $U/_logstress\
       $U/_forphan\
       $U/_dorphan\
       $U/_alarmtest\
```

```
 GNU nano 7.2                          user/user.h
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sys_sbrk(int,int);
int pause(int);
int uptime(void);

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
int atoi(const char*);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char* sbrk(int);
char* sbrklazy(int);

// printf.c
void fprintf(int, const char*, ...) __attribute__ ((format (printf, 2, 3)));
void printf(const char*, ...) __attribute__ ((format (printf, 1, 2)));

// umalloc.c
void* malloc(uint);
void free(void*);


int sigalarm(int ticks, void (*handler)());
int sigreturn(void);
```

```
 GNU nano 7.2                          user/usys.pl *
    my $prefix = "sys_";
    my $name = shift;
    if ($name eq "sbrk") {
        print ".global $prefix$name\n";
        print "$prefix$name:\n";
    } else {
        print ".global $name\n";
        print "$name:\n";
    }
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("pause");
entry("uptime");
entry("sigalarm");
entry("sigreturn");
```

```
 GNU nano 7.2                    kernel/syscall.h *
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_pause  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_sigalarm  22
#define SYS_sigreturn 23
```

```
 GNU nano 7.2                    kernel/syscall.c *
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);
extern uint64 sys_sigalarm(void);
extern uint64 sys_sigreturn(void);

// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_pause]   sys_pause,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_sigalarm]  sys_sigalarm,
[SYS_sigreturn] sys_sigreturn,
};
```

GNU nano 7.2 kernel/proc.h *

```
};

enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  struct spinlock lock;

  // p->lock must be held when using these:
  enum procstate state;        // Process state
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  int xstate;                  // Exit status to be returned to parent's wa>
  int pid;                     // Process ID

  // wait_lock must be held when using this:
  struct proc *parent;         // Parent process

  // these are private to the process, so p->lock need not be held.
  uint64 kstack;               // Virtual address of kernel stack
  uint64 sz;                   // Size of process memory (bytes)
  pagetable_t pagetable;       // User page table
  struct trapframe *trapframe; // data page for trampoline.S
  struct context context;      // swtch() here to run process
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)

  int alarm_interval;          // 0 = alarm off
  int alarm_ticks;             // ticks since last alarm
  uint64 alarm_handler;        // user address of handler
  int alarm_going;             // 1 while handler is running (blocks re-ent>
  struct trapframe alarm_tf;   // saved registers from the interrupted code


};
```

GNU nano 7.2 kernel/proc.c *

```
  }

  // An empty user page table.
  p->pagetable = proc_pagetable(p);
  if(p->pagetable == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
  }

  // Set up new context to start executing at forkret,
  // which returns to user space.
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)forkret;
  p->context.sp = p->kstack + PGSIZE;

  p->alarm_interval = 0;
  p->alarm_ticks = 0;
  p->alarm_handler = 0;
  p->alarm_going = 0;

  return p;
}
```

```
  GNU nano 7.2                    kernel/sysproc.c *
// return how many clock tick interrupts have occurred
// since start.
uint64
sys_uptime(void)
{
  uint xticks;

  acquire(&tickslock);
  xticks = ticks;
  release(&tickslock);
  return xticks;
}

uint64
sys_sigalarm(void)
{
  int ticks;
  uint64 handler;
  argint(0, &ticks);
  argaddr(1, &handler);

  struct proc *p = myproc();
  p->alarm_interval = ticks;
  p->alarm_handler = handler;
  p->alarm_ticks = 0;
  return 0;
}

uint64
sys_sigreturn(void)
{
  struct proc *p = myproc();
  *p->trapframe = p->alarm_tf;   // restore every saved register, including>
  p->alarm_going = 0;            // allow the next alarm
  return p->trapframe->a0;
}
```

```
  GNU nano 7.2                      kernel/trap.c *
    printf("            sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
    setkilled(p);
  }

  if(killed(p))
    kexit(-1);

  // give up the CPU if this is a timer interrupt.
  if(which_dev == 2){
    if(p->alarm_interval > 0 && !p->alarm_going){
      p->alarm_ticks++;
      if(p->alarm_ticks >= p->alarm_interval){
        p->alarm_ticks = 0;
        p->alarm_tf = *p->trapframe;           // snapshot of user registers
        p->alarm_going = 1;                    // don't re-enter the handler
        p->trapframe->epc = p->alarm_handler;  // return to user mode INTO >
      }
    }
    yield();
  }
```

Qemu compiles and runs:

```
mkfs/mkfs fs.img README  user/_cat user/_echo user/_forktest user/_grep user
/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stre
ssfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/
_forphan user/_dorphan user/_alarmtest  user/_call user/_bttest
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) bloc
ks 1953 total 2000
balloc: first 1019 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -
smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,i
f=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-b
us.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```